
TITANIUM_OS V1

Il sistema operativo di una vita

MATTEO BENENATI · MAGGIO 2026 · VERSIONE 1.0



Questo non è un documento tecnico. È la storia di come un saldatore ha costruito il proprio sistema operativo — una macchina CNC da 178 kg, un ecosistema AI, e un loop che si autoalimenta. Se sei un developer, troverai l'architettura. Se sei un artigiano, troverai i numeri. Se sei né l'uno né l'altro, troverai la storia.

0. Il problema

Lavoravo in 4 aziende diverse in 15 anni. Saldature TIG su titanio per i scarichi MotoGP di SCProject. Robot ESSEGI per packaging industriale. Presse DATWLER. Controllo qualità su impianti di refrigerazione LU.VE.

Ogni lavoro aveva il suo sistema, le sue procedure, le sue competenze. Io portavo tutto nella testa.

Il problema: la testa non scala. L'ADHD peggiora le cose. Ogni mattina ricominciavo da zero — dove ero rimasto, cosa avevo deciso, cosa andava fatto.

Nel 2025 ho deciso che non era un problema di volontà. Era un problema di infrastruttura.

TITANIUM_OS è la risposta.

Non è un'app. Non è un tool. È un sistema che gira da solo — che cattura, processa, ricorda e agisce al posto mio — così io posso usare l'energia per costruire cose fisiche.

1. Il fisico: V32

IL PROBLEMA

Volevo lavorare il titanio in casa. Le fresatrici CNC commerciali a 3 assi con precisione IT6-IT7 costano dai 15.000€ in su. Non li avevo.

LA SOLUZIONE

Ho costruito la mia.

V32 è una fresatrice CNC a 3 assi che sto costruendo da zero nella Taverna — il lab sotto casa. Non è un progetto hobbistico. È uno strumento di produzione.

I NUMERI

Parametro	Valore
Massa	178 kg — corpo unico, niente molle
Precisione	±0.019 mm (IT6-IT7)
Investimento	EUR 2.250
BEP	61 ore lavorate = 1,4 mesi
ROI Anno 1	322%
Tariffa	EUR 45/h (Precision Lab)

COME FUNZIONA

Struttura in acciaio S235 saldato TIG. Profilati 40×40×3 per il telaio, 60×60 per le colonne e il portale. Guide lineari, viti a ricircolo di sfere, servomotori. HMI TP900 Comfort per il controllo.

La decisione strutturale più importante: **corpo unico, nessuna molla di smorzamento**. La rigidità viene dalla geometria, non dagli ammortizzatori. Meno variabili, più precisione.

STATO ATTUALE

Config G — 65% completato. Attualmente sto saldando i rinforzi: gusset 200mm sulle colonne Z, diagonali, tiranti M10. Epoxy fill per smorzare le vibrazioni residue.

Blockers: mandrino 2.2kW ER20 da ordinare. Decisione silent blocks in valutazione (versione A senza / versione B con 18mm).

PERCHÉ ESISTE IL V32 NEL DOCUMENTO DI TITANIUM_OS

Perché il **V32** è il **primo cliente del sistema digitale**. Ogni documento tecnico, ogni decisione, ogni misura sul V32 finisce nella knowledge base. Claude conosce il V32 come se ci avesse lavorato accanto. Questo è il loop.

2. GENESIS — il sistema digitale

GENESIS è il nome dell'ecosistema software di TITANIUM_OS. Non è un software singolo. È un insieme di nodi che comunicano tra loro, girano di notte, e rendono ogni sessione di lavoro più informata della precedente.

Versione attuale: **GENESIS v1.4 — 83% completato.**

Mappa dei nodi attivi:

```
TITANIUM_OS
├─ MCP Server          - il ponte tra Claude e l'ecosistema
├─ MENTE RAG v4.0      - la memoria a lungo termine
├─ NEXUS Swarm         - orchestratore agenti paralleli
├─ Agenti AI (8)       - specialisti chiamabili on-demand
├─ Story Agent         - trasforma i commit in episodi
├─ Research Agent      - cerca su 13 fonti accademiche
├─ Computer Use        - controlla il PC da codice
├─ Dashboard v7.0     - pannello di controllo visivo
├─ API Server          - backend Flask locale
├─ n8n                 - automazioni e workflow
└─ Daily Brief         - briefing mattutino automatico
```

3. MCP Server — il cervello del sistema

COS'È UN MCP SERVER (IN 30 SECONDI)

Normalmente, Claude Code è un assistente intelligente ma cieco: non sa nulla del tuo progetto se non quello che gli mostri nella chat. MCP (Model Context Protocol) è un protocollo che permette a Claude di chiamare **strumenti personalizzati** — come se potesse aprire cassetti del tuo sistema.

È come la differenza tra chiedere a qualcuno "com'è la situazione?" e dargli accesso diretto al tuo dashboard.

IL MIO MCP SERVER

Il mio ha **8 tool** chiamabili da Claude in qualsiasi sessione:

Tool	Cosa fa
<code>get_state</code>	Legge STATE.json — milestone attivo, % completamento, blockers
<code>update_milestone</code>	Aggiunge un milestone verificato e committa
<code>search_mente</code>	Interroga il RAG — cerca nella knowledge base
<code>get_daily_brief</code>	Restituisce il briefing del giorno
<code>list_content_ready</code>	Lista episodi pronti per pubblicazione
<code>nexus_ask</code>	Manda una query all'orchestratore NEXUS
<code>rag_update</code>	Triggera rebuild incrementale del RAG
<code>update_session_context</code>	Aggiorna il contesto della sessione corrente

ESEMPIO CONCRETO

Quando apro una nuova sessione, Claude legge automaticamente

`RIAVVIO_SESSIONE.txt` — un file generato dal sistema stesso all'ultima chiusura.

Contiene: di cosa stavamo parlando, thread aperti, decisioni prese, prossima azione.

Zero ricostruzione del contesto.

4. MENTE RAG v4.0 — la memoria esternalizzata

IL PROBLEMA

Ogni settimana prendo decisioni tecniche — su materiali, geometrie, software, fornitori. Se non le documento, spariscono. Se le documento senza un sistema di recupero, è come avere una biblioteca senza catalogo.

COS'È IL RAG (IN 60 SECONDI)

RAG = Retrieval Augmented Generation.

Invece di mettere tutto il testo in una chat (impossibile — troppo lungo), lo **indicizzi** in un database vettoriale. Quando fai una domanda, il sistema recupera i pezzi più rilevanti e li passa a Claude come contesto.

È come avere un assistente che sa dove sono tutti i tuoi documenti e li tira fuori al momento giusto.

IL MIO RAG — PERCHÉ È DIVERSO

La maggior parte dei RAG usa solo ricerca semantica (ChromaDB, embeddings). Funziona bene per concetti generali, male per termini tecnici esatti: numeri di codice, nomi di componenti, misure.

Il mio usa **4 stadi in sequenza**:

Query: "silent blocks V32"

- |
 - 1. ChromaDB semantico → "trova concetti simili"
 - paraphrase-multilingual-MiniLM-L12-v2
 - 384 dimensioni, IT/EN/DE/FR
 - 2. TF-IDF BM25 keyword → "trova le parole esatte"
 - sklearn, ngram 1-2, 60.000 feature
 - 3. RRF merge (k=60) → "combina i due ranking"
 - Reciprocal Rank Fusion — nessuna calibrazione necessaria
 - 4. CrossEncoder rerank → "riordina per rilevanza vera"
 - ms-marco-MiniLM-L-6-v2
 - legge query+testo insieme, non solo vettori

Risultato: top-5 chunk più rilevanti → passati a Claude

RISULTATI

- **6.000+ chunk indicizzati** da documenti in [MENTE/](#)
- Rebuild incrementale: solo i file modificati vengono rielaborati — **<20 secondi**
- Funziona in italiano, inglese, tedesco, francese

ESEMPIO CONCRETO

Domando: "qual era la decisione sui silent blocks?" Il RAG trova il documento [V8_DELTA.md](#) scritto 3 settimane fa con la nota: "silent blocks 18mm IN VALUTAZIONE — v.A senza / v.B con". Claude la restituisce con il contesto completo.

Senza RAG: avrei dovuto ricordarmi dove l'avevo scritto, aprire il file, cercarlo.

5. Gli Agenti AI — i consulenti specializzati

COS'È UN AGENTE AI (IN 30 SECONDI)

Un agente AI è una configurazione specializzata di un LLM — un modello linguistico a cui hai dato un ruolo, un contesto, e a cui puoi fare domande come se fosse un esperto.

Non è magia. È un system prompt ben costruito + accesso ai dati giusti.

La differenza tra "chiedere a Claude" e "chiedere a un agente" è la stessa che c'è tra chiedere a un collega generico o chiamare direttamente lo specialista.

I MIEI 8 AGENTI

Agente	Specializzazione	Stato
THEMIS	Codice, architettura, analisi tecnica V32/GENESIS	Attivo
TESLA	Hardware, elettronica, CNC, IoT, motoristica	Attivo
FORGE	Meccanica, officina, saldatura TIG/MIG, MIMS	Attivo
AQUA	Idraulica, pneumatica, sistemi fluidi	Attivo
LEX	Legale, contratti, normative industriali	Attivo
SIEMENS	PLC Siemens, automazione industriale, Step 7/TIA Portal	Attivo
ARIA	ADHD scaffolding, scheduling, life OS	Attivo
EVA	WhatsApp automation, prenotazioni Vita Natura	In sviluppo

COME LI CHIAMO

Dal terminale: `ask -Agent FORGE "quale angolo di inclinazione per gusset 200mm su profilato 60x60?"`

Dall'API: `POST /api/agents/ask` con `{"agent": "FORGE", "question": "..."}`

Dalla Dashboard: AgentsView — interfaccia glassmorphism con log di ogni consultazione.

ESEMPIO CONCRETO

Stavo decidendo se usare gusset triangolari o quadrati sui rinforzi della colonna Z. Ho chiesto a FORGE. Ha risposto con calcolo di momento resistente, confronto peso/rigidità, e raccomandazione basata sul tipo di carico della mia configurazione.

Prima lo avrei cercato su YouTube per 2 ore.

6. NEXUS Swarm — quando servono più esperti insieme

IL PROBLEMA

Alcune domande non hanno una risposta che viene da un solo dominio. "Devo usare silent blocks?" coinvolge meccanica (FORGE), elettronica del motion control (TESLA), e forse analisi strutturale (THEMIS).

LA SOLUZIONE

NEXUS è l'orchestratore swarm: riceve una query, decide quali agenti coinvolgere, li esegue in parallelo, e aggrega le risposte.

COME FUNZIONA

```
Query → NEXUS
      |
      ├── ThreadPoolExecutor
      |   ├── FORGE (meccanica)
      |   ├── TESLA (elettronica)
      |   └── THEMIS (analisi)
      |
      └── Aggregazione risposte → risposta unificata
```

In parallelo, non in sequenza. Risposta in secondi, non minuti.

IL RAG GRAPH

NEXUS ha anche una mappa delle connessioni tra documenti — un grafo networkx con **114 nodi e 218 archi**. Sa che il documento sul V32 è collegato a quello su MIMS, che a sua volta è collegato a VITA_NATURA. Quando cerchi qualcosa, considera anche i documenti correlati.

7. Story Agent — la memoria narrativa

IL PROBLEMA

Ogni sessione di lavoro produce commit git. Dopo 3 mesi hai 200 commit e non sai più cosa hai costruito, perché, e in quale ordine.

LA SOLUZIONE

Story Agent gira ogni notte alle 02:07. Prende i commit git degli ultimi giorni, li raggruppa per tema (con Claude API), e genera **episodi narrativi** — racconti di quello che è stato costruito.

COME FUNZIONA

```
git log (ultimi 7 giorni)
|
▼
Raggruppamento tematico (Claude haiku - veloce)
|
├── "logging system refactor"    → Episodio 1
├── "nuove feature dashboard"   → Episodio 2
└── "fix avvio automatico"      → Episodio 3
|
▼
Generazione dual-pass (haiku bozza → sonnet finale)
|
▼
Markdown strutturato: titolo, sommario, corpo, tag, reel_hook
```

I RISULTATI

Ad oggi: **36+ episodi** divisi in serie (S0 origine, S1 costruzione, S2 sistema, MOMENTI intermezzo).

Questi episodi hanno tre usi: 1. **Memoria**: capisco cosa ho fatto senza rileggere i commit 2. **Contenuto**: base per podcast, LinkedIn, YouTube 3. **Dataset**: alimentano il fine-tuning del mio LLM personale

8. Research Agent — il bibliotecario automatico

IL PROBLEMA

Quando progetto qualcosa di tecnico (geometria gusset, ottimizzazione RAG, architettura swarm), cerco paper accademici. Farlo a mano su arXiv + Google Scholar + GitHub = 45 minuti.

LA SOLUZIONE

Research Agent cerca su **13 fonti in parallelo** con una sola query.

LE 13 FONTI

Fonte	Tipo
arXiv	Preprint CS, AI, fisica, matematica
Semantic Scholar	Grafo accademico, 200M+ paper
OpenAlex	Metadati open access
BASE	Bielefeld Academic Search Engine
GitHub	Repository di codice
Unpaywall	Versioni open-access di paper a pagamento
CrossRef	DOI e citazioni
CORE	Paper open research
PubMed	Letteratura biomedica
Internet Archive	Testi storici e digitalizzati
POLITesi	Tesi Politecnico di Milano
CNKI (parziale)	Letteratura accademica cinese
Baidu Scholar (parziale)	Ricerca cinese

Query in parallelo via asyncio. Risultati deduplicati per similarità titolo. Risposta in ~4 secondi.

9. Computer Use — il PC che si controlla da solo

IL PROBLEMA

Alcune azioni sui computer richiedono interazione con la UI — click, form, browser. Non hanno API. Richiedono un umano davanti allo schermo.

LA SOLUZIONE

Computer Use è il nodo che permette a TITANIUM_OS di controllare il desktop come se ci fossi io.

COME FUNZIONA — DUE LIVELLI

Livello 1: CDP (Chrome DevTools Protocol) Se Chrome parte con `--remote-debugging-port=9222` (attivo da oggi in START_LOGIN.bat), Playwright si connette direttamente. Accesso totale a qualsiasi tab aperta: esegui JavaScript, compila form, clicca elementi — senza toccare il mouse.

```
ba = BrowserAgent()
ba.connect_cdp()
page = ba.navigate("https://github.com/settings/profile")
page.fill("#user_profile_bio", "nuovo testo")
page.locator("button[type=submit]").first.click()
```

Livello 2: PyAutoGUI + win32gui (fallback) Per qualsiasi applicazione, non solo il browser: - Trova finestre per nome o classe - Forza il focus con `AttachThreadInput` - Invia tasti, muove il mouse, cattura screenshot

PERCHÉ È IMPORTANTE

Oggi ho usato questo sistema per aggiornare automaticamente il profilo GitHub — bio, location, creazione di 5 repository pubbliche con README bilingui. Senza aprire un browser manualmente.

Il prossimo passo: qualsiasi automazione su LinkedIn, form web, applicazioni Windows.

10. Daily Brief e Dashboard — vedere tutto in un posto

DAILY BRIEF

Ogni mattina alle 07:30, un job automatico genera `DATA/daily_brief_last.md` — un riassunto dello stato del sistema: milestone attivo, blockers, ultimi commit, contenuti pronti, alert.

Apro il PC e so già dove sono.

DASHBOARD V7.0

Interfaccia React + Vite che gira su `localhost:5173`. Sempre accesa (watchdog la rilancia se crashia).

Componenti principali: - **Pillars**: barre di progresso

V32/GENESIS/MIMS/VITA_NATURA/IDENTITY - **AgentsView**: log delle consultazioni agli agenti, interfaccia glassmorphism - **CommandBar** (Ctrl+K): ricerca rapida su tutto - **MatteoSection**: skill tree, interessi, principi, roadmap 2030 - **Dot grid**: sfondo animato

11. GitHub — la presenza pubblica

IL PROBLEMA

Il codice era tutto in un monorepo privato. Nessuna visibilità esterna. Nessun modo per altri di trovare quello che avevo costruito.

LA STRATEGIA

Tre livelli: 1. **TITANIUM_OS** (repo principale) — sistema completo, pubblico, aggiornato ogni notte 2. **5 repo standalone** — ogni nodo estratto come strumento riutilizzabile 3. **Profilo README** — chi sono, cosa costruisco, stato live del sistema

I 5 REPO STANDALONE

Repo	Problema che risolve
<code>hybrid-rag</code>	RAG ibrido locale: ChromaDB + BM25 + CrossEncoder. Senza cloud.
<code>git-narrator</code>	Trasforma i commit git in episodi narrativi. Ogni notte.
<code>multi-source-research</code>	Cerca su 13 fonti accademiche con un comando.
<code>claude-session-bridge</code>	Persistenza del contesto tra sessioni Claude Code.
<code>desktop-agent</code>	Controlla finestre, browser, app da Python. CDP-first.

Ogni README è bilingue EN+IT — inglese per visibilità globale, italiano in fondo per chi vuole approfondire.

IL PROFILO

Bio: *"Building a personal AI OS while welding a CNC machine. Same hands, different materials. 15 yrs industrial → code."* Location: Milano, Italy.

12. Il loop — come tutto si connette

Questa è la parte più importante del documento.

Non è una lista di tool. È un loop che si autoalimenta.

```
OFFICINA (fisico)
|
|— Decido qualcosa sul V32
|   ↓
|   Documento in MENTE/SESSIONI/
|   ↓
|   RAG indicizza in <20 secondi
|   ↓
|   La prossima sessione Claude sa già quella decisione
|
|— Faccio un commit git
|   ↓
|   Story Agent (02:07) genera un episodio
|   ↓
|   Episodio → dataset LLM → fine-tuning domenica notte
|   ↓
|   Il mio LLM personale conosce la mia storia
|
|— Costruisco un pezzo
|   ↓
|   Aggiungo milestone a STATE.json
|   ↓
|   GitHub profile si aggiorna automaticamente
|   ↓
|   Il mondo vede il progresso in tempo reale
```

1 input → N output. Un commit produce: episodio + dataset + milestone + storia pubblica.

13. Le persone AI

TITANIUM_OS non usa solo Claude genericamente. Ha personaggi AI con ruoli definiti.

Nome	Ruolo	Stato
THEMIS	Esecuzione tecnica, codice, analisi	Attivo
EVA	WhatsApp automation, prenotazioni Vita Natura	In sviluppo
AVA	YouTube avatar, script video, reel	Pianificato
ARIA	ADHD scaffolding, scheduling	Futuro
NEXUS	Orchestratore swarm	Attivo
TESLA	Hardware, elettronica, CNC	Attivo
FORGE	Meccanica, officina, saldatura	Attivo

EVA è il prossimo milestone critico: un bot WhatsApp per Vita Natura (il centro estetico della mia compagna Maria) che gestisce prenotazioni in automatico. Zero click per Maria. Zero telefonate gestite manualmente.

14. I numeri — solo quelli verificati

Metrica	Valore
Sessioni totali	16
Milestone verificati	33
Chunk RAG	6.000+
Episodi generati	36+
Commit nel repo	~200
Nodi GENESIS attivi	12
Tool MCP	8
Agenti AI configurati	8
Fonti Research Agent	13
Repo GitHub pubblici	6
V32 completamento	65%
GENESIS completamento	83%

15. La visione — 2030

Obiettivo: capannone artigianale proprio entro il **15 Luglio 2030**.

Non è un obiettivo lavorativo. È un obiettivo di sovranità.

Il percorso: 1. V32 → completo e in produzione → genera le prime ore fatturate 2. BEP raggiunto in 61 ore (1,4 mesi) → ROI 322% Anno 1 3. Margine reinvestito al 60% in strumenti e scala 4. VULCAN (la macchina per gli stampi) → costruito con V32 5. MIMS (modulo per tegole industriali) → primo prodotto 6. Vita Natura → EVA gestisce le prenotazioni → Maria lavora, non amministra 7. GENESIS completo → il sistema gira da solo mentre costruisco

Il capannone non è la fine. È la condizione per fare le cose senza compromessi — spazio, attrezzatura, autonomia.

Appendice — Stack tecnico

```
Linguaggi: Python 3.11 · JavaScript (React + Vite)
AI/ML:      Claude API · ChromaDB · SentenceTransformer
            CrossEncoder · TinyLlama-1.1B · LLaMA-Factory
Backend:    Flask · MCP · n8n
Frontend:    React + Vite · Tailwind · Zustand · TanStack Query
Automazione: Playwright · PyAutoGUI · win32gui · pyperclip
Hardware:    Windows 10 Pro · Getac (campo) · Acciaio S235 · Titanio
OS:          TITANIUM_OS v2.8.0
```

*TITANIUM_OS V1 — Matteo Benenati — Maggio 2026 Aggiornato automaticamente
ogni notte da TI_NightPush 04:07*

*Un sistema che gira da solo vale più di 10 abitudini che dipendono dalla
volontà.*